

Beyond Bioinformatics: Snakemake for Versatile Computational Workflows

Virtual Tutorial - ISMB/ECCB 2025

Dr Carlos Rodrigues | July 14, 2025

Research Scientist @ CSIRO

whoami

Carlos Rodrigues

Health and Biosecurity (H&B)

Australian Centre for Disease Preparedness (ACDP) @ CSIRO

mir032@csiro.au

X @carlosmr12

research.csiro.au/hostresponse



CSIRO

Schedule Overview

- **09:00-09:30am** Introduction to Snakemake and Workflow Principles
- **09:30-10:30am** Domain specific examples
 - Bioinformatics workflow (Hands-on)
 - Data Science/Machine Learning workflow (Hands-on)
- **10:45-11:45am** Advanced Snakemake features
 - Wildcards and config files
 - HPC Environments
 - Parameterised simulation workflow (Hands-on)
- **12:00-12:45pm** Practical considerations
 - Best practices
 - Q&A

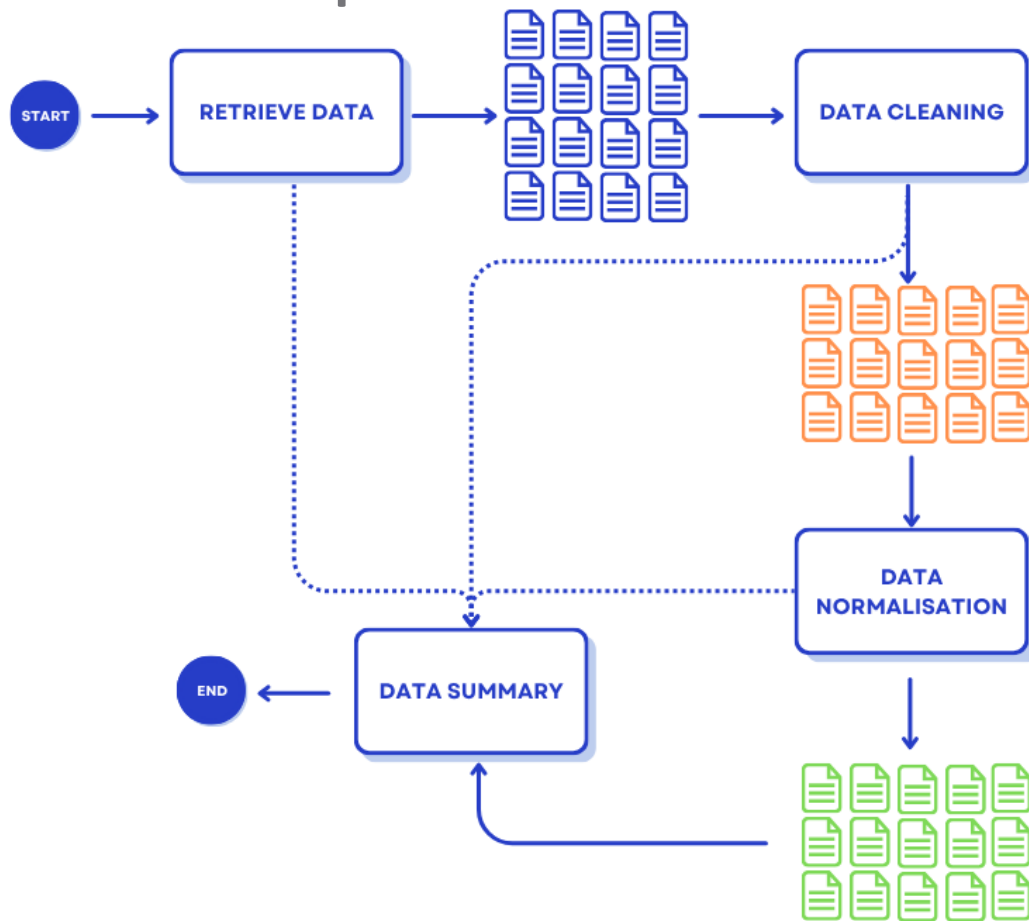
Breaks

10:30-10:45am

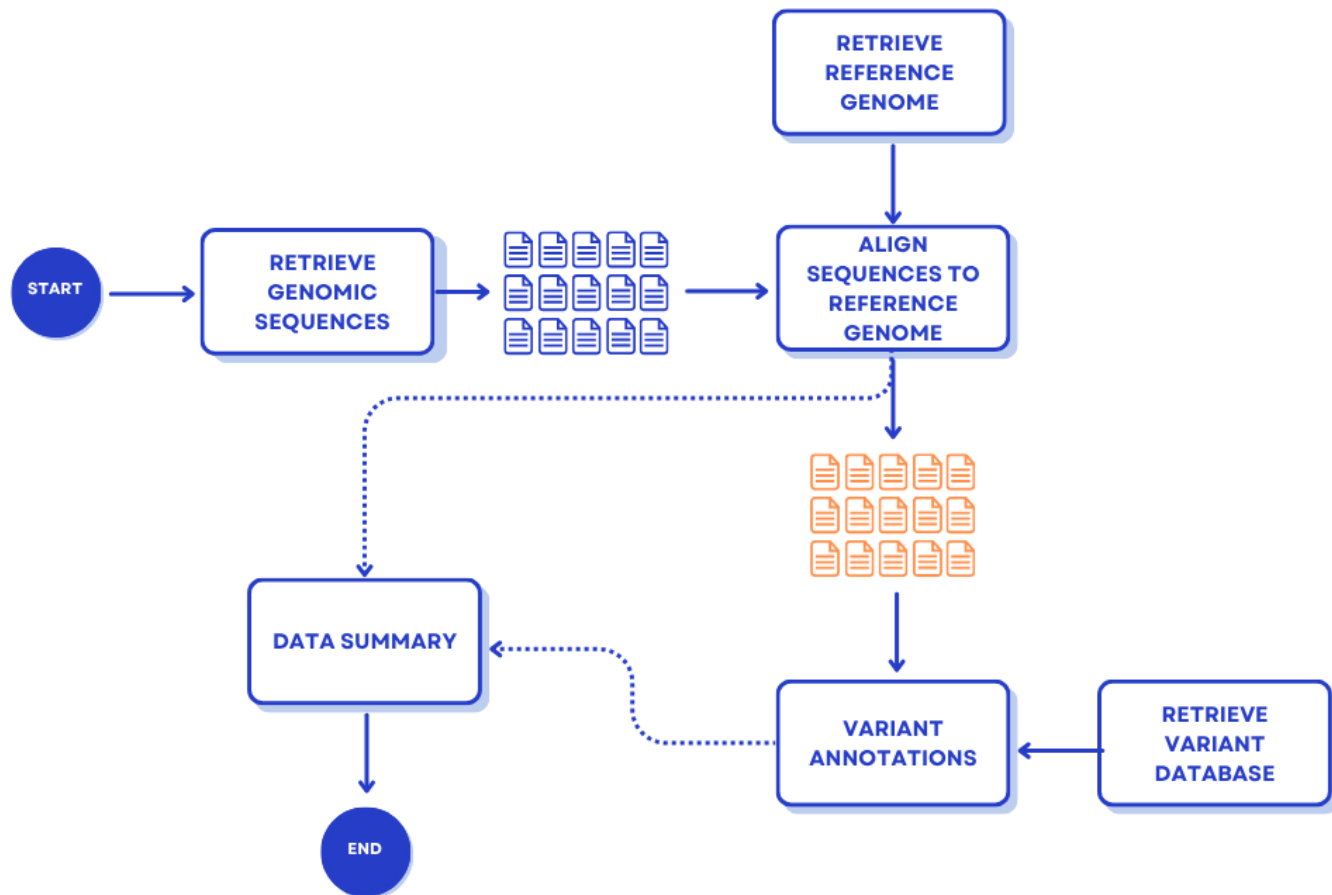
11:45-12:00pm

Introduction to Snakemake and Workflow Principles

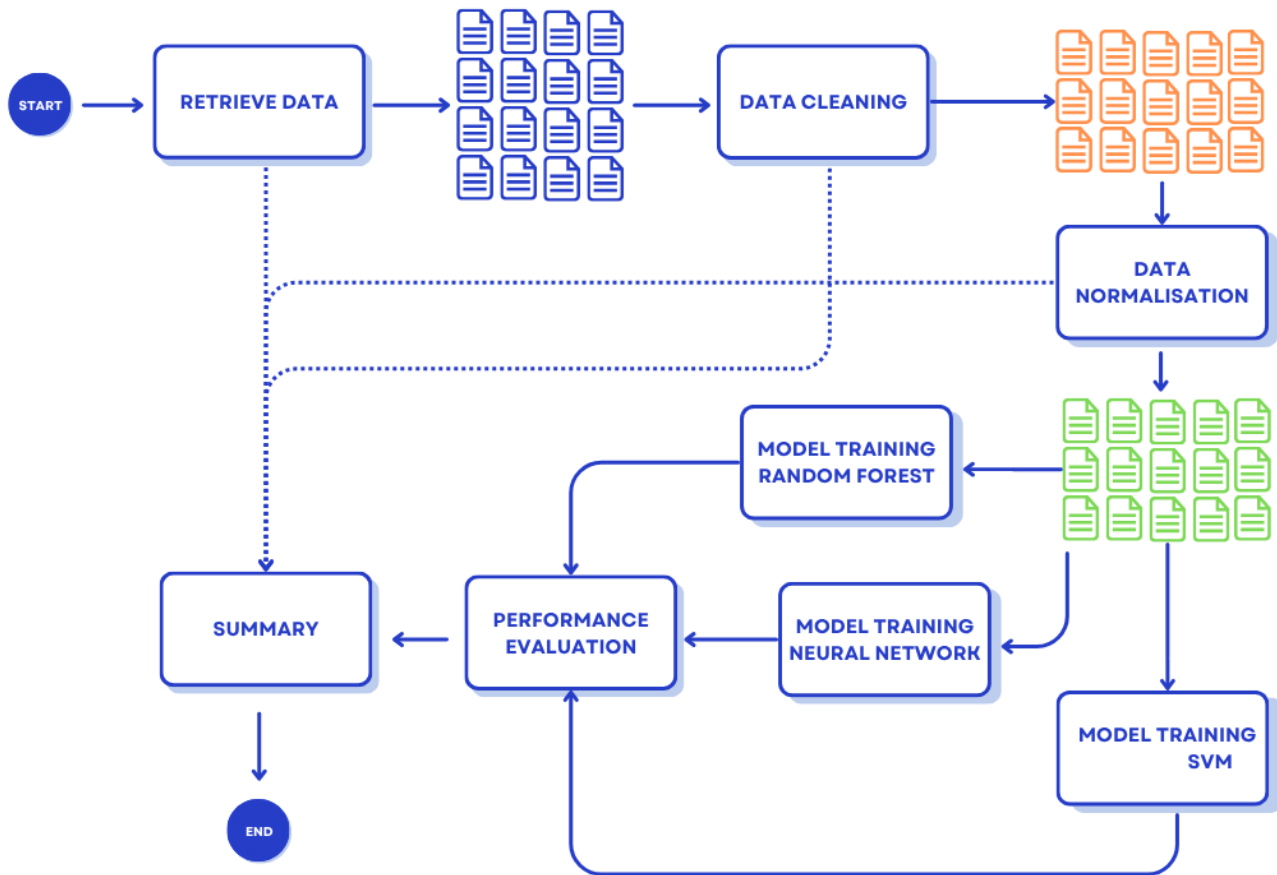
Workflows – Example 1



Workflows – Example 2



Workflows – Example 3



Snakemake - Overview

“GNU Make paradigm: workflows are defined in terms of rules that define how to create output files from input files”

- Reproducible and scalable data analysis
- Workflow are described via human readable, Python based language
- Seamlessly scaled to server, cluster, and cloud environments
- Can be turned into interactive portable browser-based reports
- Sakemake is “highly popular”
 - > 11 new citations per week
 - Mentioned in 2 Nature technology features
 - > 1M downloads on Anaconda
 - 2.5k Stars / 594 Forks on GitHub

Snakemake - Installation



- Conda/Mamba

- Full installation

```
$ conda create -c conda-forge -c bioconda -n snakemake snakemake  
$ conda activate snakemake  
$ snakemake --help
```

- Minimal installation

```
$ conda create -c conda-forge -c bioconda -n snakemake snakemake-minimal  
$ conda activate snakemake  
$ snakemake --help
```

- Pip

```
$ pip install snakemake
```

Snakemake – Today's Practice Environment

- Pre-configured at github.com/carlosmr12/snakemake-ismbeccb2025
 - Gitpod
 - VSCode

It requires a GitHub account.

- If you don't have one yet, follow the instructions below:
 1. Navigate to <https://github.com>
 2. Click **Sign up**
 3. Follow the prompts to create your personal account
 4. During sign up, you will be asked to verify your email address
 - Without a verified email address, you won't be able to complete some basic GitHub tasks, such as creating a repository



Snakemake

- Basic definitions
 - Snakefile
 - Rules
 - Name
 - Input
 - Output
 - Params
 - Shell/Run/Script
 - Bash commands
 - Plain python functions
 - Scripts

rule myrule:

input:

"path/to/inputfile",

output:

"path/to/outputfile",

params:

var1 = 5,

var2 = "tutorial"

shell:

"somecommand {params.var1} {params.var2} {input} {output}"

Snakemake - Rules

Rule to run a bash command

```
rule bash_example:
    output:
        result = "bash_output.txt"
    shell:
        "echo 'This is from bash' > {output.result}"
```

Rule to run a Python function

```
rule python_function_example:
    output:
        result = "python_function_output.txt"
    run:
        with open(output.result, 'w') as f:
            f.write('This is from Python function')
```

Snakemake - Rules

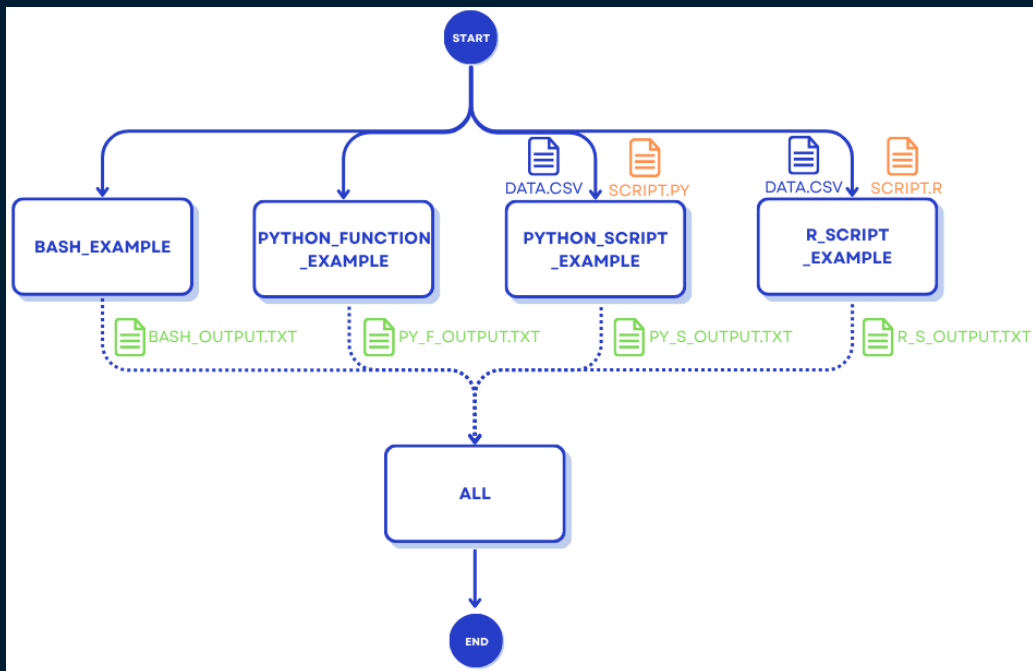
Rule to run a plain Python script

```
rule python_script_example:
    input:
        data = "data.csv"
    output:
        result = "python_output.txt"
    shell:
        "python script.py {input.data} {output.result}"
```

Rule to run an R script

```
rule r_script_example:
    input:
        data = "data.csv"
    output:
        result = "r_output.txt"
    shell:
        "Rscript script.R {input.data} {output.result}"
```

Snakemake – Rules Dependency



```

1 # Top-level rule to trigger all examples
2 rule all:
3     input:
4         "bash_output.txt",
5         "python_script_output.txt",
6         "python_function_output.txt",
7         "r_script_output.txt"
8
9 # Rule to run a bash command
10 rule bash_example:
11     output:
12         result = "bash_output.txt"
13     shell:
14         "echo 'This is from bash' > {output.result}"
15
16 # Rule to run a Python function
17 rule python_function_example:
18     output:
19         result = "python_function_output.txt"
20     run:
21         with open(output.result, 'w') as f:
22             f.write('This is from Python function')
23
24 # Rule to run a plain Python script
25 rule python_script_example:
26     input:
27         data = "data.csv"
28     output:
29         result = "python_script_output.txt"
30     shell:
31         "python script.py {input.data} {output.result}"
32
33 # Rule to run an R script
34 rule r_script_example:
35     input:
36         data = "data.csv"
37     output:
38         result = "r_script_output.txt"
39     shell:
40         "Rscript script.R {input.data} {output.result}"
  
```

Domain Specific Examples:

- Bioinformatics
- Data science/Machine Learning

Part 1

Part 2

github.com/carlosmr12/snakemake-ismbeccb2025

CSIRO

Break

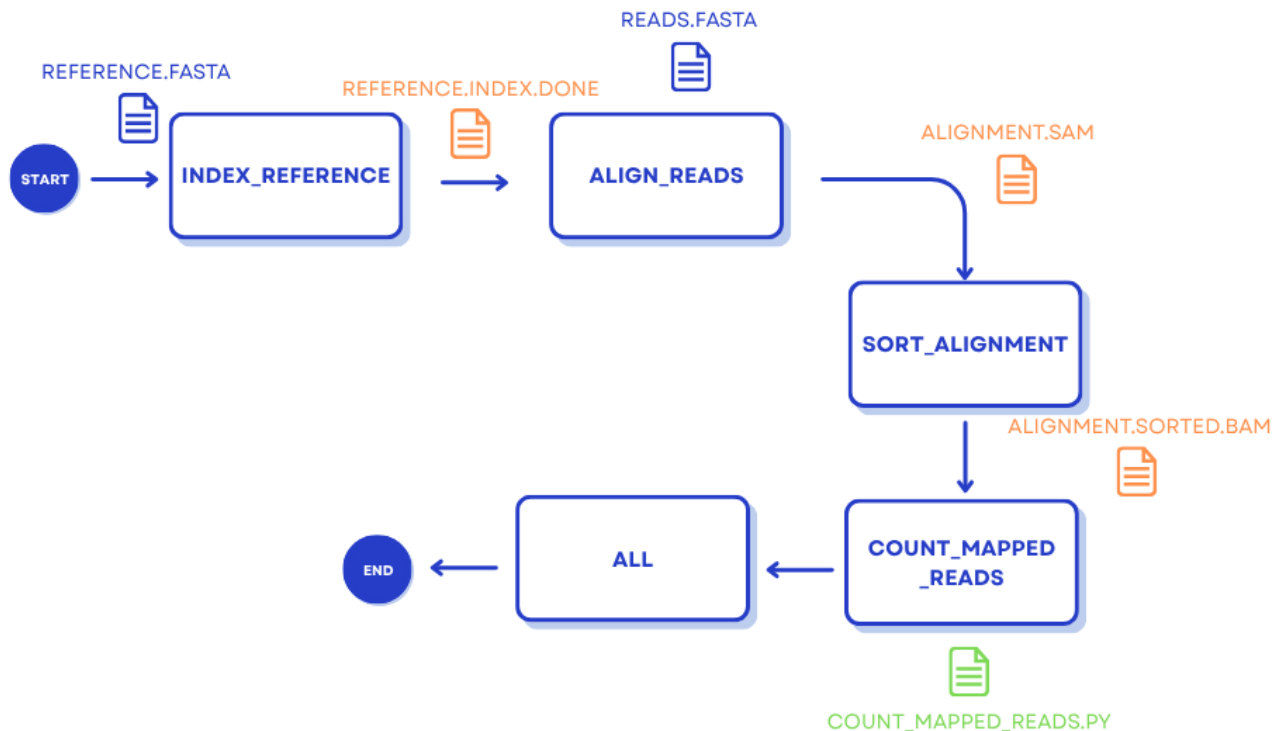
15 minutes

Advanced Snakemake Features:

- Wildcards and config files
- High Performance Computing (HPC) Environments

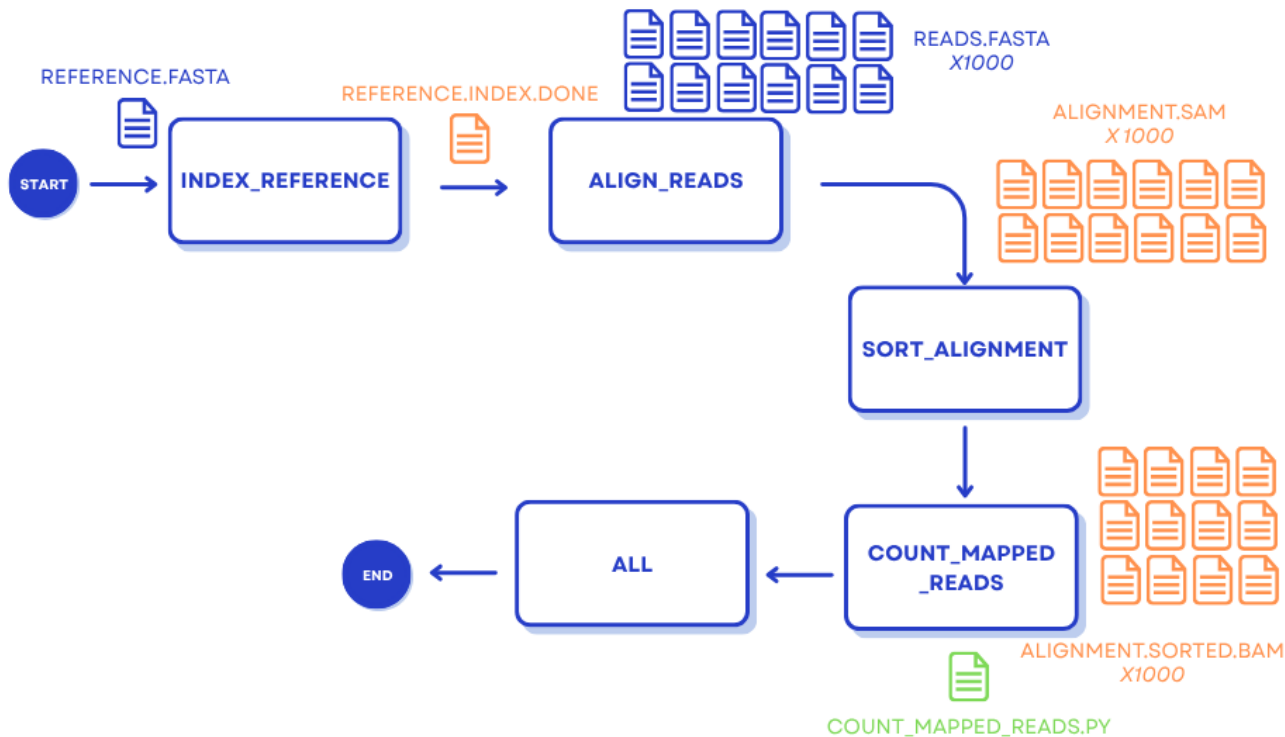
Wildcards – Generalising Rules

Our current *bioinfo_workflow.snk* – ***a single input file***



Wildcards – Generalising Rules

Real world scenario: we want to perform actions on **a lot more files**



Wildcards – Generalising Rules

Real world scenario: we want to perform actions on a lot more files

- *glob_wildcards*
 - *When the files whose names we can use to read the wildcard values*
- *expand*

```
1  from snakemake.io import glob_wildcards
2
3  # Dynamically determine the list of samples from the reads files
4  SAMPLES, = glob_wildcards("reads/{sample}.fasta")
5
6  rule all:
7      input:
8          expand("mapped_reads_{sample}.txt", sample=SAMPLES)
9
10 rule index_reference:
11     input:
12         "reference.fasta"
13     output:
14         "reference.index.done"
15     shell:
16         "bwa index {input} && touch {output}"
17
18 rule align_reads:
19     input:
20         index_done="reference.index.done",
21         reads="reads/{sample}.fasta"
22     output:
23         "alignment_{sample}.sam"
24     shell:
25         "bwa mem reference.fasta {input.reads} > {output}"
26
27 rule sort_alignment:
28     input:
29         "alignment_{sample}.sam"
30     output:
31         "alignment_{sample}.sorted.bam"
32     shell:
33         "samtools sort {input} -o {output}"
```

Config

- params

```
32 | rule filter_alignment:
33 |     input:
34 |         "alignment.sorted.bam"
35 |     output:
36 |         "alignment.filtered.bam"
37 |     params:
38 |         quality_threshold=20
39 |     shell:
40 |         "samtools view -q {params.quality_threshold} -b {input} > {output}"
```

- Config via command line

```
$ snakemake --snakefile bioinfo_workflow.snk --cores 1 --config QUALITY_THRESHOLD=20
```

```
32 | rule filter_alignment:
33 |     input:
34 |         "alignment.sorted.bam"
35 |     output:
36 |         "alignment.filtered.bam"
37 |     params:
38 |         quality_threshold=config['QUALITY_THRESHOLD']
39 |     shell:
40 |         "samtools view -q {params.quality_threshold} -b {input} > {output}"
```

Config

- params

```
32 | rule filter_alignment:
33 |     input:
34 |         "alignment.sorted.bam"
35 |     output:
36 |         "alignment.filtered.bam"
37 |     params:
38 |         quality_threshold=20
39 |     shell:
40 |         "samtools view -q {params.quality_threshold} -b {input} > {output}"
```

- Config via YAML file

config.yml

```
QUALITY_THRESHOLD: 20
SAMPLES: ["reads/sample1.fasta", "reads/sample2.fasta", "reads/sample3.fasta",
"reads/sample4.fasta", "reads/sample5.fasta", "reads/sample6.fasta", "reads/sample7.fasta",]
```

```
$ snakemake --snakefile bioinfo_workflow.snk --cores 1 --configfile config.yml
```

Parallel Execution (HPC)

config.yml

```
QUALITY_THRESHOLD: 20
```

```
$ snakemake --snakefile bioinfo_workflow.snk --cores 32 --configfile config.yml
```

```
1  from snakemake.io import glob_wildcards
2
3  # Dynamically determine the list of samples from the reads files
4  SAMPLES, = glob_wildcards("reads/{sample}.fasta")
5
6  rule all:
7      input:
8          expand("mapped_reads_{sample}.txt", sample=SAMPLES)
9
10 rule index_reference:
11     input:
12         "reference.fasta"
13     output:
14         "reference.index.done"
15     shell:
16         "bwa index {input} && touch {output}"
17
18 rule align_reads:
19     input:
20         index_done="reference.index.done",
21         reads="reads/{sample}.fasta"
22     output:
23         "alignment_{sample}.sam"
24     shell:
25         "bwa mem reference.fasta {input.reads} > {output}"
26
27 rule sort_alignment:
28     input:
29         "alignment_{sample}.sam"
30     output:
31         "alignment_{sample}.sorted.bam"
32     shell:
33         "samtools sort {input} -o {output}"
```

Parallel Execution (HPC)

- *threads* and *--jobs*
 - Jobs that on their own run using multiple cores

config.yml

```
QUALITY_THRESHOLD: 20
```

```
$ snakemake --snakefile bioinfo_workflow.snk --cores 32 --jobs 8 --configfile config.yml
```

```
1 from snakemake.io import glob_wildcards
2
3 # Dynamically determine the list of samples from the reads files
4 SAMPLES, = glob_wildcards("reads/{sample}.fasta")
5
6 rule all:
7     input:
8         expand("mapped_reads_{sample}.txt", sample=SAMPLES)
9
10 rule index_reference:
11     input:
12         "reference.fasta"
13     output:
14         "reference.index.done"
15     shell:
16         "bwa index {input} && touch {output}"
17
18 rule align_reads:
19     input:
20         index_done="reference.index.done",
21         reads="reads/{sample}.fasta"
22     output:
23         "alignment_{sample}.sam"
24     threads: 4
25     shell:
26         "bwa mem reference.fasta {input.reads} > {output}"
27
28 rule sort_alignment:
29     input:
30         "alignment_{sample}.sam"
31     output:
32         "alignment_{sample}.sorted.bam"
33     shell:
34         "samtools sort {input} -o {output}"
```


Advanced Snakemake Features:

- Parameterised simulation workflow

github.com/carlosmr12/snakemake-ismbeccb2025

CSIRO

Break

15 minutes

Practical Considerations:

- Best practices
- Q&A

Best Practices

- Code quality (“*linter*”)

```
$ snakemake --snakefile bioinfo_workflow.snk --lint
```

- Keep filenames short, but informative

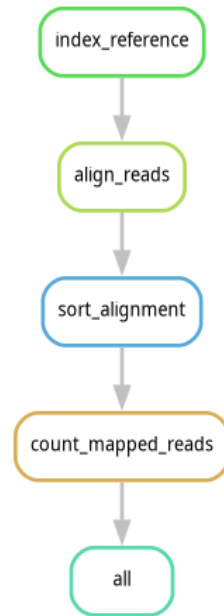
- Avoid too many special characters
- Keep consistency (e.g. _ or -)

- Keep Python code (*helper functions*) separate from rules

- Help non-experts to read the workflow

- Workflow visualisation

```
$ snakemake --snakefile bioinfo_workflow.snk --rulegraph | dot -T png > workflow.png
```



Other Useful Features

- Modularity (*include* & *subworkflow*)

- Recycling rules from other workflows

```
1  from snakemake.io import glob_wildcards
2
3  # Dynamically determine the list of samples from the reads files
4  SAMPLES, = glob_wildcards("reads/{sample}.fasta")
5
6  # Define the subworkflow for indexing the reference genome
7  subworkflow index_workflow:
8      |   workdir:
9      |       "index_dir"
10     |   snakefile:
11     |       "index_workflow.smk"
12
13     # Include per-sample processing rules
14     include: "rules.smk"
15
16     rule all:
17         |   input:
18         |       expand("mapped_reads_{sample}.txt", sample=SAMPLES)
```

Other Useful Features

- Protected and Temporary Files

- `protected` == locked (write permissions removed)
- `temp` == file will be deleted as soon as any rules depend on it are run

```
44 rule count_mapped_reads:
45     input:
46         | "alignment_{sample}.sorted.bam"
47     output:
48         | temp("mapped_reads_{sample}.txt")
49     script:
50         | "count_mapped_reads.py"
51
52 rule generate_report:
53     input:
54         | expand("mapped_reads_{sample}.txt", sample=SAMPLES)
55     output:
56         | protected("report.txt")
57     run:
58         with open(output[0], "w") as report:
59             for infile in input:
60                 # Extract sample name from filename (assumes format: mapped_reads_{sample}.txt)
61                 sample = infile.split("_")[2].split(".")[0]
62                 with open(infile) as f:
63                     count = f.read().strip()
64                     report.write(f"{sample}: {count}\n")
```

github.com/carlosmr12/snakemake-ismbeccb2025

CSIRO

Q&A



Carlos Rodrigues

Health and Biosecurity (H&B)

ACDP @ CSIRO

mir032@csiro.au

X @carlosmr12

research.csiro.au/hostresponse

Share your Feedback!

Scan the QR code to let us know your thoughts on this tutorial. Thank you!

